# Multi-Modal Machine Learning Approach for Threat Detection in WhatsApp and Telegram: Spam Links, Document Based Malware, Chat Behavior, and Sensitive Content Analysis

Detection and Classification of Spam URL-Based Messages in WhatsApp and Telegram Using a Sandbox Environment

25-26J-065

Project Proposal Report

| Student Name | Student Registration Number |
|---|---|
| GANGODA G.W.A.P. | IT22594440 |

B.Sc. (Hons) in Information Technology Specializing in Cyber Security

Sri Lanka Institute of Information Technology Sri Lanka

August 2025

## Declaration

I declare that this is my own work, and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously publish or written by another person expect where the acknowledgement is made in the text.

| Name | Student ID | Signature |
|---|---|---|
| Gangoda G.W.A.P. | IT22594440 | |

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

.....

Date: 27/08/2025

Signature of the supervisor:
(Mr. Deemantha Siriwardana)

.....

Date: 27/08/2025

Signature of the supervisor:
(Miss. Ayesha Wijesooriya)

## Table of Contents

I. INTRODUCTION ..... 8

III.1 Background and Literature Review ..... 8

III.2 Research Gap ..... 9

III.3 Research Problem ..... 10

II. OBJECTIVES ..... 11

III.1 Main Objective ..... 11

III.2 Sub-Objectives ..... 11

III. METHODOLOGY ..... 12

III.1 Requirement Gathering ..... 12

III.3.1 Past Research Analysis ..... 13

III.2 Feasibility Study ..... 14

III.3.2 Technical Feasibility ..... 14

III.3 PROJECT REQUIREMENTS ..... 15

III.3.1 Functional Requirements ..... 15

III.3.2 Non-Functional Requirements ..... 16

III.4 COMMERCIALIZATION ..... 18

IV. BUDGET AND BUDGET JUSTIFICATION ..... 19

# List of Figures

**Figure 1 - System Diagram** ..... 12
Figure 2 – Grant chart ..... 17
Figure 3 - Work Breakdown Chart ..... 17

## List of Tables

Table I-1 - Research Gap in tabular view ..... 10
Table IV-1 - Budget Analysis ..... 19

## List of Abbreviations

| | | |
|---|---|---|
| API | - | Application Programming Interface |
| CERT | - | Computer Emergency Response Team |
| GUI | - | Graphical User Interface |
| ML | - | Machine Learning |
| OS | - | Operating System |
| URL | - | Uniform Resource Locator |
| UI | - | User Interface |
| APK | - | Android Package Kit |
| CSV | - | Comma-Separated Values |

# I. INTRODUCTION

## III.1 Background and Literature Review

Recent Messaging applications such as WhatsApp and Telegram have become the most widely used digital communication tools across the world. In Sri Lanka and many other countries, people of all ages—from school children to business professionals—use these platforms daily to share messages, links, images, and documents. The popularity of these applications comes from their ease of use, speed, and ability to connect people instantly. However, this heavy reliance on messaging apps has also created new opportunities for cybercriminals to exploit users. One of the most common and dangerous threats faced by messaging app users today is the distribution of spam and malicious URLs (links).

When a user receives a suspicious link in a chat, there is always the risk that clicking on it will lead to harmful consequences. Such links may redirect the user to **phishing websites** designed to steal login credentials, **malware download pages** that compromise device security, or fake promotional sites that trick users into revealing personal or financial information. These threats are even more dangerous because they often look genuine at first glance, and ordinary users may not have the technical knowledge to recognize them. Children and non-technical users are especially vulnerable, as they can be easily deceived by seemingly harmless links promising free gifts, games, or rewards.

At present, most security measures for link detection are based on **blacklists** (databases of already known malicious links) or **static analysis** methods that check URL patterns. While these methods are useful, they have clear limitations. Attackers frequently generate **new links** or modify (obfuscate) existing ones to bypass blacklists. Static methods are also unable to detect dynamic behaviors, such as when a link redirects multiple times before reaching its final malicious destination. Another problem is that most advanced detection systems are designed for desktop computers or enterprise firewalls, which are not suitable for ordinary mobile users who depend on WhatsApp and Telegram. This leaves a **significant gap** in mobile security: there is no simple, lightweight tool that allows end users to test a suspicious link **before clicking it**.

This research is aimed at closing that gap by developing a **sandbox-based mobile application** that detects and classifies suspicious URLs shared in WhatsApp and Telegram. The idea is simple but

powerful. If a user receives a link and feels unsure about its safety, they can **copy and paste the link into the mobile app** we are developing. The application will then open the link inside a **secure sandbox environment**. This sandbox acts as a "safe container" or **protected browser**, where the link's behavior can be observed without any risk to the actual device. Even if the link tries to download malware, redirect to multiple sites, or execute malicious scripts, all of that activity happens inside the sandbox, completely isolated from the user's phone.

The sandbox will work together with **machine learning models** that analyze both the **textual properties of the URL** (such as length, domain type, number of redirects, presence of suspicious words or characters) and its **runtime behavior inside the sandbox** (such as attempted downloads, suspicious script execution, or redirection to known phishing domains). By combining these features, the system can classify the link into categories such as **Safe, Spam, or Malicious**. This classification result will be presented to the user through a **simple, user-friendly interface** that provides a clear warning before the user decides whether to open the link further.

The implementation of this system will follow a series of steps:

1. **Dataset Collection** – Collect a large set of URLs, including safe, spam, and malicious ones, from publicly available sources such as phishing datasets, Kaggle repositories, and CERT advisories. Synthetic datasets will also be generated for testing.
2. **Sandbox Design** – Create a lightweight sandbox browser within the mobile app to safely open and monitor submitted URLs. This will simulate the behavior of opening the link in a real environment without exposing the user's device.
3. **Feature Extraction** – Record features from both the **URL text** (domain name structure, special characters, link length, HTTPS usage, etc.) and the **sandbox activity** (redirections, downloads, suspicious scripts, etc.).
4. **Machine Learning Classification** – Train and evaluate machine learning models (e.g., Random Forest, Logistic Regression, or Neural Networks) to classify the URLs based on the extracted features.
5. **User Interface Integration** – Develop a mobile-friendly interface that allows users to paste suspicious links and receive an easy-to-understand safety verdict. The interface will also include the option to open the link within the sandbox browser if the user still wants to proceed.

6. **Testing and Evaluation** – Test the system with different scenarios, including phishing links, spam promotions, and benign links, to measure accuracy, efficiency, and usability.

This approach is different from traditional security mechanisms because it provides **dynamic analysis on a mobile platform**, something that is usually limited to enterprise security systems. By empowering end users to analyze links themselves, the project promotes greater awareness and safer communication.

In summary, this research addresses a very real and growing problem in today's digital communication landscape. Spam and malicious links in WhatsApp and Telegram pose significant risks to users, yet current solutions are either too limited, too static, or not mobile-friendly. The proposed sandbox-based mobile application provides a **lightweight, user-controlled, and intelligent detection system** that fills this gap. By combining sandbox isolation with machine learning classification, the system will deliver accurate and safe link verification in a simple format accessible to ordinary users. This innovation has the potential not only to enhance personal security on messaging platforms but also to contribute to the broader field of **mobile cybersecurity** by offering a new way to protect against phishing and malicious content.

## III.2 Research Gap

While existing research on malicious and spam URL detection has demonstrated strong classification accuracy and promising machine learning approaches, there are notable gaps that remain unaddressed. Research in [1] proposes a Flask-based web application for detecting malicious URLs using machine learning and NLP techniques. Although the system achieves high accuracy, it is limited to a browser-based environment and does not focus on mobile-first messaging applications such as WhatsApp and Telegram. Importantly, it does not provide users with an isolated environment to safely open suspicious links.

Research in [2] presents an effective multi-class classification framework using Random Forest, LightGBM, and XGBoost for categorizing URLs into phishing, malware, and defacement classes. While it advances the detection side with strong modeling techniques, it does not integrate with end-user messaging contexts, nor does it address the challenge of real-world obfuscated or shortened links typically found in chat platforms. Furthermore, the absence of a secure sandbox mechanism leaves users vulnerable when they attempt to interact with potentially harmful links.

Research in [3] explores spam and non-spam URL detection using traditional machine learning algorithms and feature engineering. Although it contributes valuable insights into spam classification, the work is largely dataset-driven, lacking deployment in mobile ecosystems and overlooking the need for a user-controlled, on-demand verification process for suspicious links.

From these studies, one clear gap emerges: the absence of a **mobile-first, sandbox-enabled solution** that allows messaging app users to safely test and classify suspicious URLs before clicking them. Current approaches either focus solely on offline detection accuracy or deploy as web-based tools, without addressing the user journey in WhatsApp and Telegram where the majority of such threats occur.

The proposed research aims to fill this gap by delivering a lightweight mobile application that integrates machine learning–based classification of URLs with a sandboxed safe browser. This ensures that users can not only receive an accurate classification verdict but also open links in an isolated environment without exposing their devices to harm. By addressing messaging context, sandbox security, and mobile usability together, this project provides a more comprehensive and practical

solution than those found in existing work.

*Table I-1 - Research Gap in tabular view*

| Research Paper | Messaging-App Context (WhatsApp/Telegram) | Mobile App (User-triggered, not real-time) | Sandboxed "Safe Browser" to open links | Dataset aligned to chat-shared URLs | Deployment style |
|---|---|---|---|---|---|
| **Research [1]: *Malicious URL Detection using NLP, ML & Flask*** | Not implemented (mentions WhatsApp only as a source example) | Not Implemented (Flask web UI) | Not Implemented | Not implemented (Kaggle set; generic) | Flask web app |
| **Research [2]: *Malicious URL Detection & Classification Analysis (RF/LightGBM/ XGBoost)*** | Not Implemented | Not implemented | Not Implemented | Not implemented (Kaggle; 651,191 URLs) | Research pipeline (no end-user app) |
| **Research [3]: *Spam & Non-Spam URL Detection (ML Approach*** | Not Implemented | Not implemented | Not Implemented | Not implemented (general URL sets; engineered features) | Lab prototype (no mobile/web sandbox) |
| **Proposed Solution** | **Implemented**: user pastes WhatsApp/Telegram link into the app | **Implemented**: **manual** check; no real-time background scanning (your constraint) | **Implemented**: open inside your **sandboxed safe browser** | **Implemented/Planned**: emphasize shorteners, obfuscation, multilingual chat URLs | ***Mobile app*** *+ (optionally) lightweight AP* |

## III.3 Research Problem

Current spam and malicious URL detection mechanisms used in messaging platforms are largely based on static blacklist approaches or traditional rule-based systems. While these methods can block known threats, they are inefficient against newly generated, obfuscated, or dynamically changing links, which are common in platforms like WhatsApp and Telegram. Furthermore, many existing solutions are designed for desktop environments or enterprise systems, and are often computationally heavy, unsuitable for mobile devices, and not directly accessible to end users. This gap becomes more critical as users frequently receive suspicious links through messaging apps, and have no safe way to test or verify them before clicking. A user who mistakenly opens a malicious link risks exposing personal data, downloading malware, or compromising their device. This, in turn, increases the demand for lightweight, user-friendly detection mechanisms that can operate on mobile platforms without consuming excessive resources. One promising direction is to integrate a sandbox-based environment into a mobile app, allowing suspicious links to be safely opened and analyzed in isolation. This approach ensures that any harmful behavior triggered by the link remains contained, and the user can receive an accurate classification before interacting further.

# II. OBJECTIVES

## III.1 Main Objective

The main aim of this research is to design and develop a lightweight mobile application that can detect and classify spam and malicious URLs shared through WhatsApp and Telegram. The proposed system will integrate a sandbox-based secure browser, allowing users to safely test suspicious links before interacting with them, thereby enhancing user protection and ensuring safer communication on these widely used messaging platforms..

## III.2 Sub-Objectives

This will be achieved by:

1. Studying in detail existing URL detection and classification techniques, including machine learning approaches and their limitations in mobile messaging contexts.
2. Designing and implementing a custom sandbox environment within a mobile application to safely open and analyze the behavior of suspicious links without risking the user's device.
3. Collecting and preprocessing datasets of benign, spam, phishing, and malicious URLs — with an emphasis on short, obfuscated, and chat-shared links common in WhatsApp and Telegram.
4. Extracting and engineering features from both the URL text (lexical/structural properties) and its sandbox runtime behavior (redirections, downloads, script activity).
5. Training and evaluating machine learning models to classify URLs into categories such as Safe, Spam, Phishing, or Malicious with high accuracy and efficiency.
6. Integrating the detection and classification system into a user-friendly mobile interface, enabling on-demand link verification and safe browsing functionality.
7. Testing and validating the proposed solution across different usage scenarios to ensure its effectiveness, lightweight performance, and usability in real-world mobile environments.

# III. METHODOLOGY

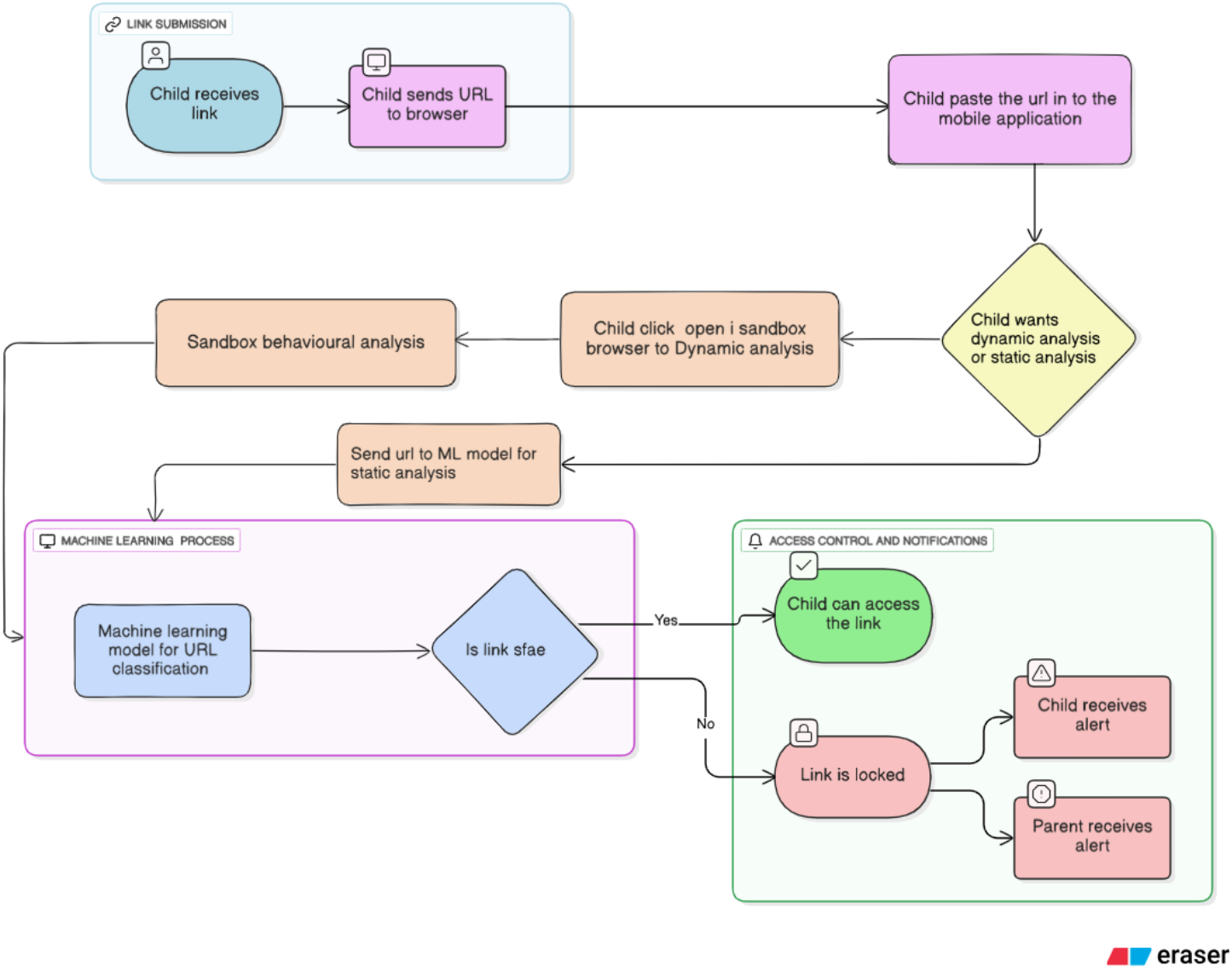

**Figure 1 - System Diagram**

The proposed system is designed to protect users from malicious or spam URLs received via messaging platforms such as WhatsApp and Telegram. It provides a structured workflow combining static feature extraction, sandbox-based behavioral analysis, and machine learning classification to ensure safe browsing. The process works as follows:

1. **Suspicious URL Reception** – The user receives a URL through WhatsApp or Telegram. Instead of clicking the link directly, the system encourages the user to verify it first for safety.

2. **Forwarding to Security App** – The user copies the suspicious URL and pastes it into the Mobile Security Application, which serves as the central verification tool.

3. **Preprocessing & Validation** – Before deeper inspection, the system parses the URL structure, normalizes its format, and prepares it for consistent analysis. This helps remove noise and ensure uniform input.

4. **Feature Extraction & Sandbox Analysis** –

   - *Static Feature Extraction:* The system inspects URL length, domain age, abnormal characters, HTTPS validation, embedded tokens, and lexical patterns to identify known suspicious indicators.

   - *Sandbox Environment:* If static checks are insufficient, the URL is opened inside a secure sandbox. Here, the system monitors DNS resolution, network traffic, redirection patterns, and embedded script execution to reveal hidden malicious activity.

5. **Machine Learning Analysis** – Both static and dynamic features are combined and fed into trained ML models (e.g., Random Forest, Logistic Regression, Neural Networks). This stage improves detection accuracy by leveraging predictive patterns.

6. **Classification Decision** – Based on ML outcomes, the system provides a safety assessment of the URL:

   - **SAFE** – Low risk detected; safe to access.

   - **SPAM** – Suspicious link; proceed with caution.

   - **MALICIOUS** – High risk detected; do not proceed.

7. **Optional Safe Browser Execution** – If a URL is flagged as spam or risky, the system offers an optional “Safe Browser” mode. Here, the link opens inside a secure, isolated

environment, preventing any compromise of the user's device.

8. **Protected User Experience** – Finally, the system communicates the results to the user clearly, helping them make informed decisions. Even if risky URLs are accessed, the device remains protected by sandbox isolation and proactive detection.

This methodology integrates **static analysis, sandbox behavior monitoring, and machine learning classification** into a lightweight mobile security solution. By combining multiple detection layers, it provides users with **real-time protection against spam, phishing, and malicious URLs** while maintaining a smooth and secure browsing experience.

## III.1 Requirement Gathering

The requirement gathering process for this research involved a combination of **literature review**, **analysis of existing systems**, and **consultation with industry experts and end-users**. The objective of this stage was to gain a deep understanding of the current methods of malicious URL detection, their limitations in messaging platforms, and the expectations of users who will interact with the proposed system.

- **Extensive Literature Review**:
  A wide range of academic papers, technical reports, and case studies were reviewed to identify the strengths and shortcomings of existing URL detection systems. Studies focusing on machine learning and natural language processing for malicious URL classification were analyzed to understand which algorithms and feature engineering techniques perform best. Special attention was paid to research using datasets from platforms like Kaggle, as well as works highlighting the use of Random Forest, Logistic Regression, and Neural Networks for URL-based spam and phishing detection. This review confirmed that while high accuracy is achievable in lab environments, there is a lack of mobile-first solutions tailored for WhatsApp and Telegram. Furthermore, no study integrated a **sandbox-based safe browser** for opening suspicious links, which represents a critical gap that our project addresses.

- **Analysis of Existing Systems**:
  Commercial security solutions such as antivirus software, URL scanners (like VirusTotal), and browser extensions were examined. These systems often rely on centralized blacklists or resource-intensive static analysis, making them unsuitable for lightweight, on-demand use on mobile devices. Moreover, these tools are not directly embedded within messaging workflows, requiring users to switch applications or copy-paste URLs into third-party platforms. This analysis reinforced the need for a dedicated, **lightweight mobile application** that seamlessly supports messaging users.

- **Stakeholder Consultation**:
  To align the project with real-world needs, consultations were conducted with cybersecurity professionals, software developers, and regular messaging app users. Cybersecurity experts emphasized the increasing sophistication of phishing links, including the use of shortened URLs, Unicode characters, and multiple redirections, which cannot always be detected by simple static methods. Software developers provided input on the technical feasibility of implementing a **sandbox environment** within a mobile app, highlighting challenges such as resource consumption, privacy, and efficiency. Additionally, feedback was collected from frequent WhatsApp and Telegram users, including students and office workers, who reported feeling uncertain about identifying suspicious links and expressed the need for a simple "check-before-you-click" tool. These insights directly shaped the user-focused design requirements of the system.

- **Workshops and Surveys**:
  Informal workshops and small-scale surveys were carried out among university students and young professionals to gather expectations for a user-friendly

interface. Respondents highlighted the importance of clarity in results (e.g., categorizing a link as Safe, Spam, or Malicious instead of complex scores) and requested the option to open links safely in a secure environment if they still wanted to access them. These findings validated the inclusion of a **sandboxed safe browser** as a core component of the proposed solution.

### III.3.1 Past Research Analysis

The analysis of prior research provides valuable insights into existing solutions for malicious URL and spam detection and highlights several gaps that justify the need for this proposed study:

#### III.2.2-i Literature Review of Key Studies:

- **Research [1]: (Malicious URL Detection using NLP, Machine Learning and Flask)**

  This study integrates Natural Language Processing (NLP) methods like TF-IDF, Count Vectorizer, and Hashing Vectorizer for extracting URL features, then applies classifiers such as Random Forest and Logistic Regression. The model achieves high accuracy (up to 97.5%) and is deployed as a Flask-based web application. However, its scope is limited to browser-based testing. It does not consider sandbox environments for runtime behavioral analysis, nor does it address URLs shared in encrypted messaging platforms such as WhatsApp or Telegram.

- **Research [2]: (URL Spam Detection Using Machine Learning Classifiers)**

  This work explores multiple machine learning classifiers for distinguishing spam from non-spam URLs, demonstrating robust accuracy. While effective for spam filtering, the study is constrained by its focus on static datasets and general classification. It does not address dynamically generated or obfuscated malicious URLs that can bypass static detection, and it lacks mechanisms tailored for real-time scanning in messaging platforms**.**

- **Research [3]: (Towards Fighting Cybercrime: Malicious URL Attack Type Detection using**

  Multiclass Classification)
  This research moves beyond binary classification and identifies different malicious URL attack types using multiclass classification. It improves understanding of various threat categories. However, the study is more academic and resource-intensive, making it less suited for lightweight, mobile-friendly applications. Moreover, it does not integrate sandboxing techniques or account for constraints in real-time messaging ecosystems like WhatsApp and Telegram.

#### III.2.2-ii Identification of Gaps

- **Sandbox-Based Behavioral Analysis**: While existing studies use ML and NLP techniques to classify URLs, none of them incorporate a sandbox environment to safely execute and observe suspicious URLs before classification..
- **Messaging App Integration**: Current research focuses on generic datasets and web-

based tools, overlooking the specific risks of spam URLs shared through WhatsApp and Telegram.

- **Dynamic and Evasive URLs**: Static approaches (lexical/NLP feature extraction) alone cannot reliably detect obfuscated or dynamically generated spam URLs.
- **User-Friendly Mobile Solutions**: Although one study developed a Flask-based app, no work has focused on creating lightweight, mobile-first solutions tailored for messaging app users.

## III.2 Feasibility Study

The technical feasibility of the proposed research assesses whether the tools, technologies, and techniques required for the development and implementation of a sandbox-based spam URL detection system are appropriate.

### III.2.2-i Tools and Technology

- **Python:** Will serve as the core programming language for implementing ML models, sandbox integration, and backend services.
- **Android Studio / Flutter**: For building the proposed mobile application interface, allowing users to submit suspicious links and access results.
- **Scikit-learn & TensorFlow:** Libraries for training and testing machine learning models for URL classification.
- **Docker-based Sandbox Environment**: To safely analyze the behavior of URLs in an isolated container without impacting the host device.
- **SQLite / Firebase**: Lightweight databases for storing URL classification logs and user reports.

### III.2.2-ii Algorithm and Architectures

- **Machine Learning Models:** Random Forest, Logistic Regression, and Naïve Bayes will be applied, as validated by prior studies, to classify URLs**.**

- **NLP Feature Extraction:** Techniques such as TF-IDF and Hashing Vectorizer will be used to extract meaningful patterns from URLs.

- **Behavioral Monitoring in Sandbox:** Suspicious URLs will be opened in the isolated browser, and activities like redirections, script executions, or download attempts will be logged for analysis

### III.2.2-iii Techniques

- **Data Preprocessing:** Cleaning and feature extraction from collected datasets of URLs.
- **Sandbox Simulation:** Execution of suspicious links within a controlled environment to capture malicious behavior.
- **Classification and Evaluation:** Testing classifiers with both static (lexical) and dynamic (sandbox behavior) features to improve detection accuracy.
- **Performance Evaluation:** Measuring accuracy, precision, recall, and execution time to ensure effectiveness in mobile scenarios.

## III.3 PROJECT REQUIREMENTS

### III.3.1 Functional Requirements

- **URL Submission:** Users should be able to input suspicious WhatsApp/Telegram links into the app.
- **Sandbox Execution:** The system must safely open links in a sandbox browser to observe malicious activity.
- **Classification:** The model must classify URLs as safe, spam, or malicious based on both static and dynamic analysis.
- **Safe Browser Access:** Provide a secure, sandbox-based browser for users to open links without exposing their device to threats.

### III.3.2 Non-Functional Requirements

- **Accuracy:** High detection accuracy for spam and malicious URLs.
- **Usability:** Lightweight mobile app with simple interface for non-technical users.
- **Performance:** Efficient execution without heavy resource consumption.
- **Security:** Strong isolation through sandboxing to prevent system compromise.
- **Scalability:** Ability to incorporate additional datasets and detection models in the future.

## III.4 WORK BREAKDOWN AND GRANT CHART

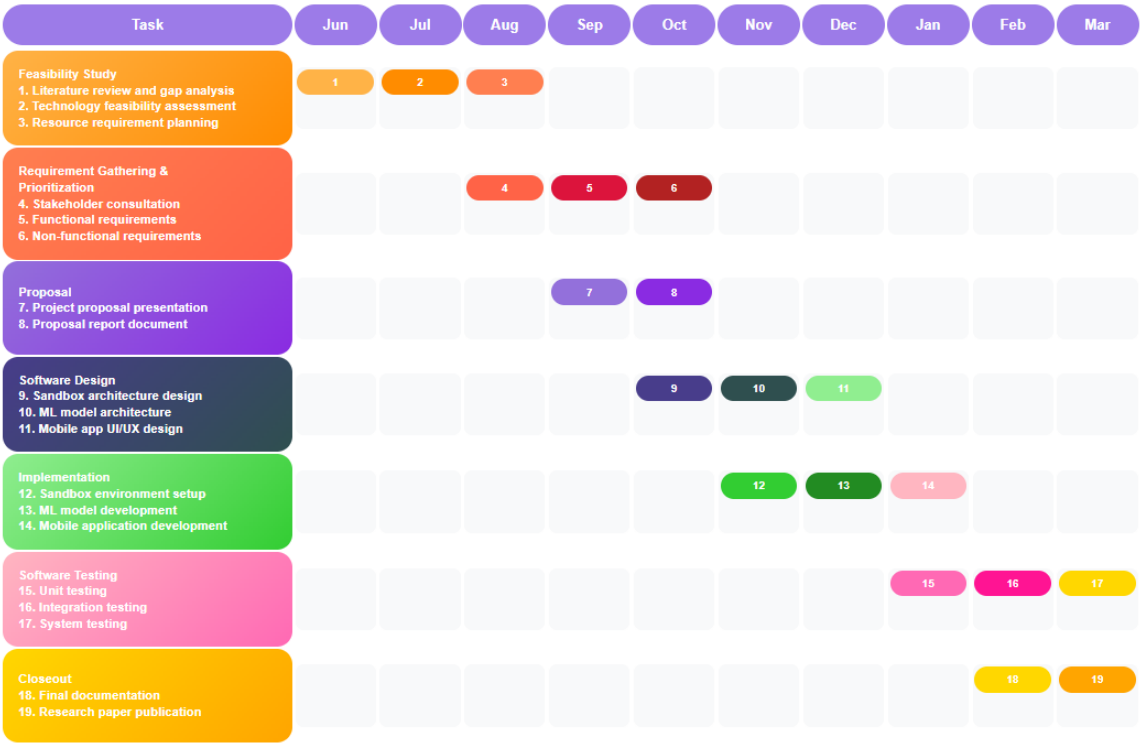


Figure 2 – Grant chart

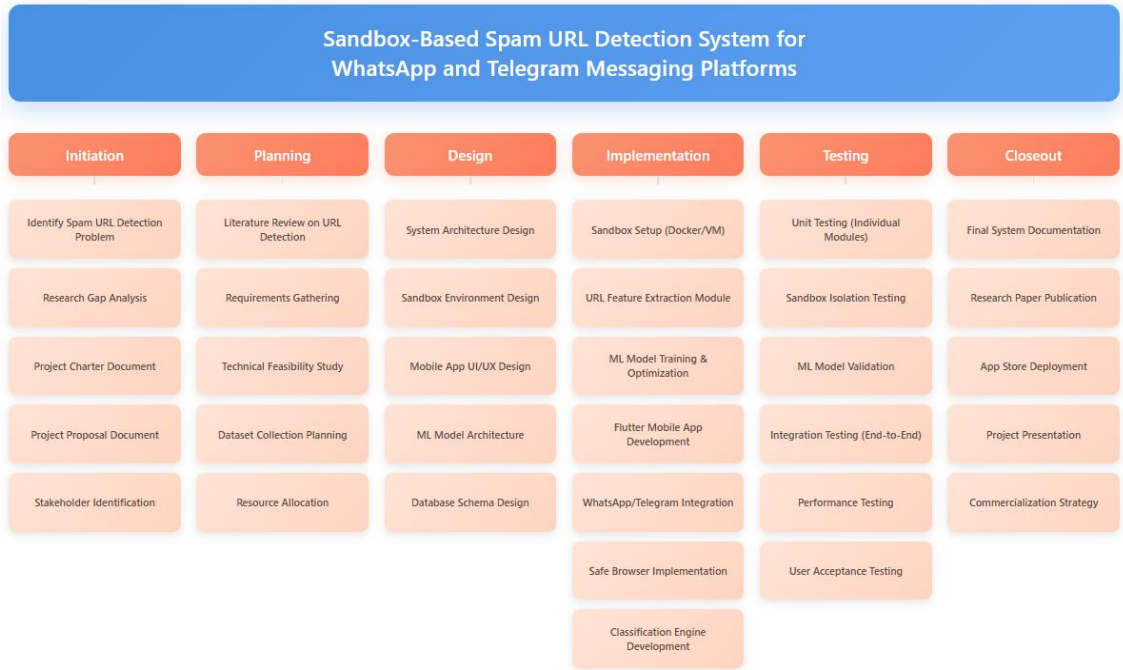


Figure 3 - Work Breakdown Chart

## III.5 COMMERCIALIZATION

The commercialization strategy for the proposed system is designed to ensure affordability, accessibility, and scalability while meeting the needs of different user groups ranging from individual parents to schools and child safety organizations.

**Pricing Packages** are structured in three tiers to provide flexibility and match diverse requirements. The **Basic package ($15)** focuses on essential protection by offering URL and document analysis. This is particularly suitable for users who need fundamental security features at an affordable cost. The **Pro package ($30)** expands functionality by including URL, document, and chat analysis, addressing a broader set of risks that children face in encrypted messaging environments. The **Premium package ($50)** is the most comprehensive, covering all features such as URL, document, chat, and media (images/videos) analysis, alongside the Parent Dashboard for real-time alerts and monitoring. This tier is ideal for families and institutions that require full visibility and advanced safety features.

**Availability** of the application is ensured through multiple trusted platforms. The system will be distributed via the **Google Play Store** for Android users, the **Apple App Store** for iOS devices, and an **official website download option** to allow direct installations. This multi-platform approach enhances user convenience and ensures wide adoption across different device ecosystems.

The **Promotion Strategy** combines both digital marketing and institutional partnerships. Targeted **social media campaigns** on platforms such as Facebook, Instagram, and TikTok will raise awareness among parents and guardians. **Targeted advertisements** will specifically focus on parents, schools, and NGOs concerned with child safety. In addition, partnerships with **child safety organizations** will build credibility and encourage wider adoption. To foster trust and demonstrate effectiveness, the project will also provide **demo versions and trial packages** for schools, enabling institutions to evaluate the system before subscribing.

Through these combined approaches, the commercialization plan ensures that the solution is not only technically effective but also market-ready, sustainable, and impactful in protecting children online.

**Key Commercialization Points:**

- **Target Market**

    1. **General Messaging App Users**: Everyday users of WhatsApp and Telegram, especially young adults, students, and non-technical individuals who are most vulnerable to phishing scams.
    2. **Educational Institutions**: Schools, universities, and online learning platforms that rely heavily on WhatsApp/Telegram communication with students.
    3. **Corporate Sector and SMEs**: Small and medium businesses that depend on messaging apps for client communication but lack dedicated IT security tools.
    4. **Cybersecurity Service Providers**: Security companies and ISPs who can bundle the app as part of their digital safety offerings.

**Unique Selling Proposition:** Unlike existing URL detection tools that rely solely on static analysis or centralized blacklists, this solution integrates **dynamic sandbox testing** and **real-time machine learning classification** into a **single mobile platform**. The ability to safely open suspicious links inside a sandboxed browser is a unique differentiator that reduces risk significantly. Furthermore, its design as a lightweight, user-initiated system ensures it works efficiently without draining battery life or consuming excessive resources, making it especially suitable for mobile-first regions.

- **Revenue Model**

    1. **Freemium Model**: A free version with basic URL classification, supported by advertisements or limited feature access. Premium plans could unlock unlimited link checks, detailed risk reports, and sandbox access.

2. **Subscription Service**: Monthly or yearly subscription packages for individual users who want ad-free, full-featured protection.
3. **Enterprise Licensing**: Offering the application to organizations (schools, businesses, NGOs) under bulk licensing agreements.
4. **Partnerships with Telecom Providers and Messaging Platforms**: Collaborating with ISPs or cybersecurity firms to integrate the app as part of mobile data packages or bundled digital safety services.

- **Market Entry Strategy:** The initial market entry can focus on **regions with high WhatsApp/Telegram penetration**, such as South Asia, the Middle East, and Latin America. Early adoption campaigns can be conducted in **universities and schools**, where digital safety awareness is increasingly emphasized. Collaborations with **cybersecurity awareness programs** and **digital literacy NGOs** can also boost credibility and trust. Later, expansion can target corporate clients and partnerships with tech companies offering cybersecurity suites. A strong emphasis will be placed on **educational marketing**, demonstrating how the app prevents real phishing attacks and malware infections, thereby encouraging users to see it as an essential digital safety tool.

Altogether, given the current and future demand for safe car-to-car and car-to-infrastructure communication, as well as the provision of a novel, all-in-one mobile platform for this project, it has a high likelihood of being successfully commercialized. Given the increasing risks associated with spam and malicious links on WhatsApp and Telegram, the proposed solution offers a **novel, accessible, and practical approach** to user safety. Its combination of **sandbox-based security, machine learning classification, and mobile-first design** provides a unique edge in the cybersecurity market. With the right commercialization strategy — targeting both individual users and organizations, supported by flexible revenue models and strong market entry campaigns — the solution has a **high likelihood of successful adoption and long-term sustainability**.

# IV. BUDGET AND BUDGET JUSTIFICATION

The total estimated budget for the project is LKR 125,000, carefully distributed across essential categories to ensure effective development, testing, and dissemination.

No cost is allocated for Software and Tools, as the project utilizes open-source technologies such as Python, Scikit-learn, TensorFlow Lite, and Kotlin/Flutter, which provide advanced functionalities without licensing expenses. For Sandbox and Cloud Testing, LKR 25,000 is allocated to use services like AWS or GCP, enabling secure simulation of malicious URL executions and scalable model training.

To improve detection accuracy, LKR 12,000 is set aside for Threat Intelligence APIs, covering paid services such as VirusTotal, PhishTank, and Hybrid Analysis for validation. Mobile Test Devices account for the largest cost, LKR 55,000, for two mid-range Android phones, which will be used for real-time testing and integration of sandbox features.

Dataset Handling is allocated LKR 8,000 to cover preprocessing, secure storage, and management of large datasets needed for training models. Publication and Dissemination requires LKR 15,000 for journal or conference submissions, posters, and printing. Finally, Miscellaneous expenses of LKR 10,000 are included to cover internet, electricity, contingencies, and backups.

*Table IV-1 - Budget Analysis*

| Category | Description | Estimated Cost (LKR) |
|---|---|---|
| **Software & Tools** | Python, Scikit-learn, TensorFlow Lite, Kotlin/Flutter (all open-source). | 0 |
| **Sandbox / Cloud Testing** | Cloud VMs (AWS/GCP) for simulating URL sandbox execution & model training. | 25,000 |
| **Threat Intelligence APIs** | Paid API calls (VirusTotal, PhishTank, Hybrid Analysis) for validation/testing. | 12,000 |
| **Mobile Test Devices** | 2 Android phones (mid-range) for sandbox browser integration testing. | 55,000 |
| **Dataset Handling** | Collection, preprocessing & secure storage of large URL datasets. | 8,000 |
| **Publication & Dissemination** | Conference/journal submission fees, posters, printing. | 15,000 |
| **Miscellaneous** | Internet, electricity, contingency, backups. | 10,000 |
| **Total** | | **125,000** |

## Summary

This project proposal aims to strengthen the security of messaging platforms such as WhatsApp and Telegram by developing a lightweight mobile application for the detection and classification of spam and malicious URLs. The system addresses the limitations of existing solutions, which often rely on static blacklists or are not designed for mobile messaging contexts. The proposed solution integrates machine learning–based classification with a sandbox environment, allowing users to safely test suspicious links before interacting with them. A detailed literature review and stakeholder consultations will inform the design, while datasets of benign, spam, phishing, and malicious URLs will be used for model training and evaluation. The final application will feature an intuitive user interface, delivering accurate classifications and the option to open links securely in a sandboxed safe browser. Furthermore, commercialization strategies will be explored, with potential markets including individual users, educational institutions, and organizations seeking lightweight, user-friendly cybersecurity solutions for messaging environments.

# V. References


[1] Sahoo, D., Liu, C., & Hoi, S. C. (2017). Malicious URL Detection using Machine Learning: A Survey. arXiv preprint arXiv:1701.07179. Available: https://arxiv.org/abs/1701.07179

[2] Alsharnouby, M., Alaca, F., & Chiasson, S. (2015). Why phishing still works: User strategies for combating phishing attacks. *International Journal of Human-Computer Studies*, 82, 69-82.

[3] Rao, R. S., & Pais, A. R. (2021). Lexical features based malicious URL detection using machine learning techniques. *Materials Today: Proceedings*, 46(20), 10479-10484. doi:10.1016/j.matpr.2021.01.293

[4] Zhou, Y., Liu, Q., Zhang, J., & Feng, Q. (2022). An effective detection approach for phishing websites using URL and HTML features. Scientific Reports, 12, 8842. https://doi.org/10.1038/s41598-022-10841-5

[5] Kim, T., Park, N., Hong, J., & Kim, S. W. (2022). Phishing URL Detection: A Network-based Approach Robust to Evasion. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. Los Angeles, CA, USA.

[6] Nguyen, H. T., Tran, V. H., & Le, K. M. (2023). Enhancing Security in WhatsApp: A System for Detecting Malicious and Inappropriate Content. In *Proceedings of the 12th International Symposium on Information and Communication Technology*. ACM Digital Library.

[7] Zhang, L., Wang, Y., & Chen, X. (2024). PhishingRTDS: A real-time detection system for phishing attacks using a Deep Learning model. Computers & Security, 141, 103785. https://doi.org/10.1016/j.cose.2024.103785

[8] Al-Ahmadi, S., & Alshahrani, H. (2023). Classification of Malicious URLs Using Machine Learning. Scientific Reports, 13, 15771. https://doi.org/10.1038/s41598-023-42629-5